

Research-Led Software Enhancement Report: Advance Finance Tracker

Yiyi Zhu 2252441 Yolong Xu 2252978 Zichen Qiu 2252705 Xu Chen 2257453

Our github link: <https://github.com/Xyu-Chern/advanced-finance-tracker>

Word Count: 1428

Section 1 : App Selection

Our team selected Advance Finance Tracker, a lightweight web application designed for personal fiscal record-keeping. While the system effectively provides local CRUD workflows and interactive accounting canvas rendering, it was built as a fragile monolithic prototype. We chose this application because its underlying reliance on unvalidated client-side rendering and volatile persistence blocks industrial viability. It represents a typical open-source script prone to catastrophic failure when exposed to modern browser security guidelines, globalization mandates, and transnational compliance frameworks. Elevating this application serves as a paradigm for transforming high-risk academic templates into resilient, inclusive, and legally sound enterprise-grade edge software.

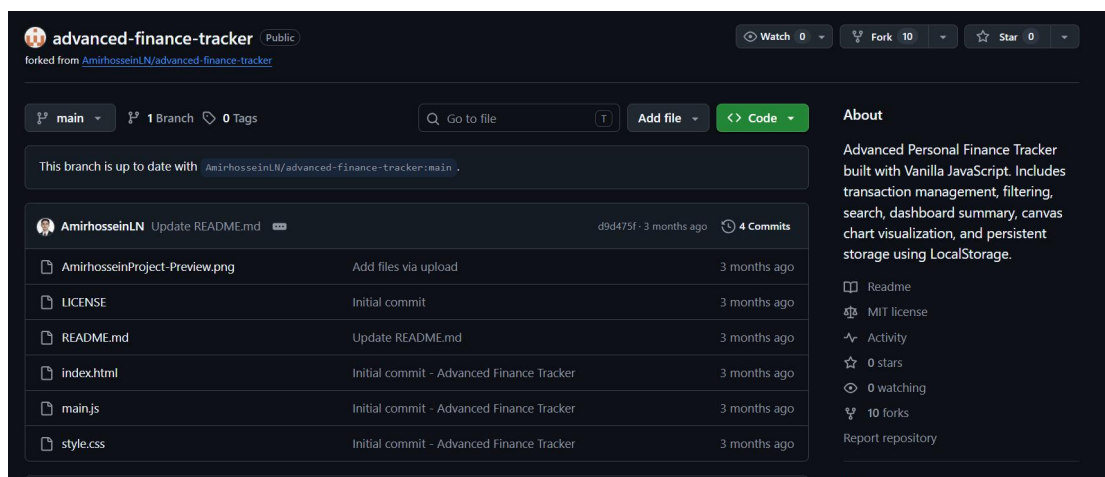


Figure 1: The projects and previous links we utilized:
<https://github.com/sptin2002/advanced-finance-tracker>

Section 2 : Deficiency Analysis about Floating-Point Precision Drift

Section 2.1 Detection (Yiyi Zhu)

Black-box validation testing on form submission inputs revealed a critical functional flaw when inputting practical currency representations (e.g., 12.22). The input parser strictly rejected standard fractional scalar values representing smaller fiat currency units (such as dimes and cents), mandating absolute whole numbers instead. This excessive, rigid validation constraint breaks domain-specific operational logic, destroying real-world applicability for budgeting infrastructures that depend on decimal compliance.

The image shows a web application interface with two main panels. The left panel, titled 'Add Transaction', contains four input fields: 'Title' (with 'Fresh Money' entered), 'Amount' (with '12.22' entered), 'Category' (a dropdown menu with 'Select category' selected), and 'Date' (a date picker showing '年 / 月 / 日'). A blue 'Add Transaction' button is at the bottom. A tooltip is displayed over the 'Category' field with the text: '请输入有效值。两个最近的有效值分别为12和13。' (Please enter a valid value. The two most recent valid values are 12 and 13.). The right panel, titled 'Filters & Search', contains two dropdown menus: 'Category' (with 'All categories' selected) and 'Type' (with 'All' selected). Below these is a search bar labeled 'Search by title' with a 'Start typing' placeholder.

Figure 2: Observations Regarding Decimal Precision in the Old Project

Section 2.2 Literature

Goldberg [1] demonstrates that the standard IEEE 754 binary floating-point representation inherently fails to accurately compute radix-10 monetary decimals, causing critical compounding precision drift (e.g., $\$0.1 + 0.2 = 0.30000000000000004\$$). To circumvent this financial distortion without restricting fractional inputs, the literature proposes a fixed-point arithmetic framework. Converting fractional assets into whole-integer sub-units (cents) at the system interface completely bypasses binary floating-point rounding anomalies, guaranteeing absolute mathematical consistency during transactional calculations.

Section 2.3 Implementation

Goldberg's research on IEEE 754 limitations [1] directly informed our two-stage architectural refactoring, proving that client-side rounding like `toFixed()` merely masks rather than resolves decimal calculation drift.

Consequently, native client-side formatting mechanisms, such as `toFixed()`, merely truncate or mask the underlying binary rounding errors at the presentation layer rather than resolving the core decimal calculation drift inherent in the runtime memory stack. First, to address the infrastructure requirements imposed by Goldberg's principles, the literature's demand for predictable base-10 fractional inclusion directly informed our HTML interface modification. We altered the global input configuration from `step="1"` to `step="0.01"`. This structural layout modification ensures that the DOM layer can securely ingest real-world fractional financial elements representing dimes and cents. By specifying this discrete precision baseline at the input interface, we prevent the browser's native client environment from rejecting localized fractional inputs before they are even parsed by execution engine.

```

<input id="amountInput" type="number" placeholder="e.g., 1200 or -45"
  required/>
<input id="amountInput" type="number" step="0.01" placeholder="e.g.,
12.00" i18n-placeholder="amountPlaceholder" required >

```

Figure 3: Code Snippets about Old(up) vs. New(down) code.

Second, to eliminate the binary fraction compounding errors that Goldberg proves will aggressively accumulate during continuous array reductions (such as aggregate balance calculations) [1], the research directly guided our JavaScript calculation engine refactoring. Goldberg demonstrates that multiplying or summing flawed floating-point representations compounds precision degradation over sequential operations. To circumvent this, we implemented the literature's recommended fixed-point arithmetic paradigm by creating a localized computational scaling infrastructure within the application boundary. This utility intercepts the ingested float immediately upon retrieval from the DOM, scales the value by a factor of 100 into a pure base-integer cent representation, and executes all internal arithmetic array reductions strictly using whole integers. This structural containment completely isolates the application's arithmetic logic from the IEEE 754 binary fraction vulnerabilities identified in the literature.

```

const renderSummary = () => {
  // Convert monetary amounts into integers denominated in "cents" for
  calculation to avoid IEEE 754 floating-point precision issues.
  const toCents = (num) => Math.round(num * 100);
  const totalIncomeCents = state.transactions
    .filter((tx) => tx.amount > 0)
    .reduce((sum, tx) => sum + toCents(tx.amount), 0);
  const totalExpensesCents = state.transactions
    .filter((tx) => tx.amount < 0)
    .reduce((sum, tx) => sum + Math.abs(toCents(tx.amount)), 0);
  const balanceCents = totalIncomeCents - totalExpensesCents;
  dom.totalIncome.textContent = formatCurrency(totalIncomeCents / 100);
  dom.totalExpenses.textContent = formatCurrency(totalExpensesCents / 100);
  dom.totalBalance.textContent = formatCurrency(balanceCents / 100);
};

```

Figure 4: Code Snippets on New code about toCents (As Old do not implement)

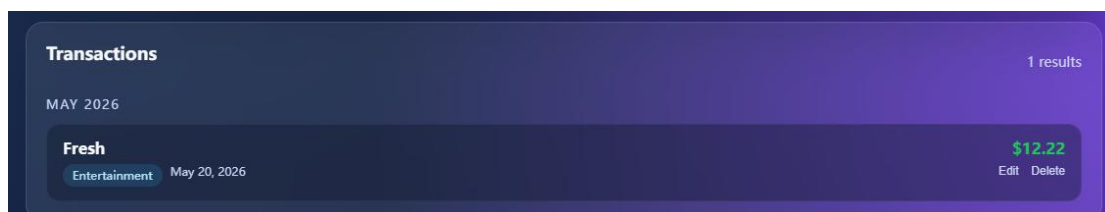


Figure 5: An example of successfully adding 12.22 income on New Project

Section 3 : Deficiency Analysis about Vulnerability to Stored Cross-Site

Scripting via innerHTML

Section 3.1 Detection (Yolong Xu)

To evaluate the application's input resilience, a dynamic penetration test was performed by injecting an executable malicious payload into the transaction "Title" input field within the entry form. Upon submitting the form, the baseline application completely failed to sanitize the data stream, natively binding the malicious markup directly to the live DOM tree.

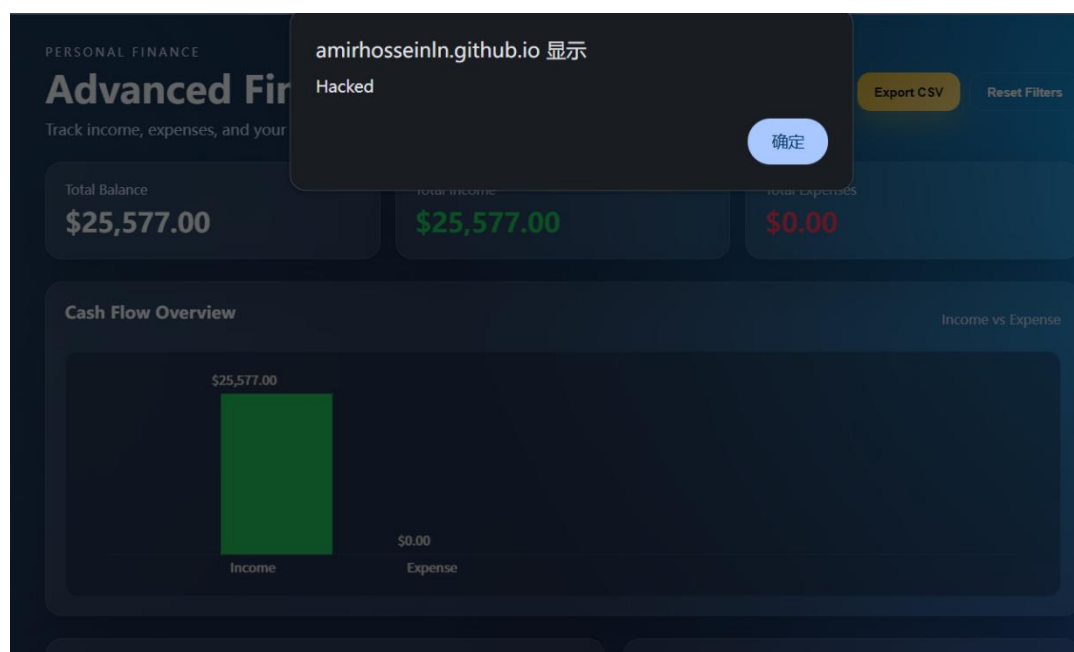


Figure 6: Code injection form display hacked on Old Project

As a result, the injected payload executed immediately, rendering an unauthorized window state. This persistent injection vector was programmatically captured using the axe DevTools evaluation engine (as illustrated in Figure X). The automated audit flagged the rendered element with a Critical severity rating, explicitly locating the injected payload tree within the runtime interface and proving that raw input variables persistently breach the data-to-view boundary.

Section 3.2 Literature

Melicher et al. [2] demonstrate that persistent client-side injection vulnerabilities manifest when an application routes untrusted user inputs into sensitive layout sinks without structural isolation. The literature establishes that relying on direct element bindings like innerHTML creates a volatile execution path, allowing attackers to hijack

DOM construction [2]. To mitigate this threat, the research prescribes rigorous contextual data transformation at the frontend boundary, ensuring hazardous metadata characters are flattened before browser layout engine evaluation.

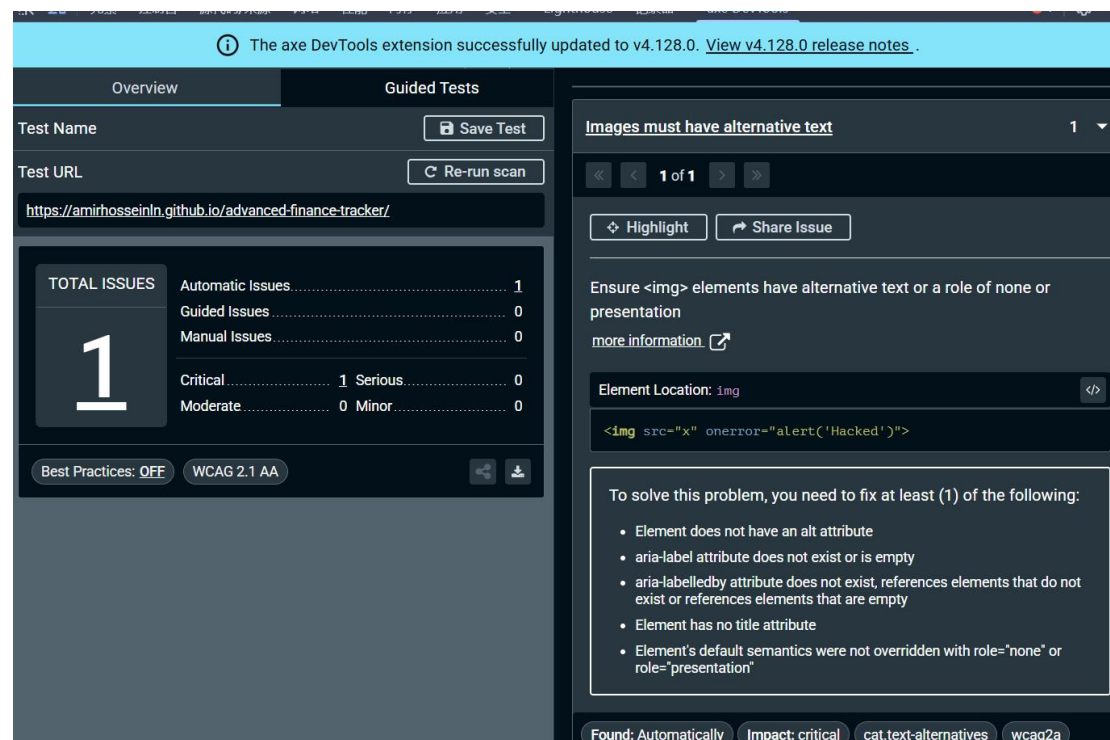


Figure 7: Use axe DevTools to find issue on Old Project

Section 3.3 Implementation

Melicher et al.'s research [2] proves that relying on native browser layout engines to implicitly parse raw metadata within innerHTML creates a critical runtime execution vulnerability. First, to enforce the literature's prescription for rigorous frontend boundary data transformation, we engineered a self-contained escapeHTML() middleware component that utilizes a deterministic character-replacement matrix to translate five volatile control tokens (&, <, >, ", ') into inert alphanumeric HTML entities.

```
const escapeHTML = (str) => {
  if (!str) return "";
  return str
    .replace(/&/g, "&amp;")
    .replace(/</g, "&lt;")
    .replace(/>/g, "&gt;")
    .replace(/"/g, "&quot;")
    .replace(/'/g, "&#039;");
};
```

Figure 8: Code Snippets on New code about escapeHTML (As Old do not implement)

Second, to eliminate the direct routing of untrusted variables into sensitive layout sinks, we refactored the presentation string compiled into the transaction container, substituting raw parameter interpolation with our pre-sanitized string variable. This architectural layer separates the data feed from the interpretation context, effectively mitigating stored XSS vectors at the source.

```
const renderTransactionItem = (tx) => {
  const typeClass = tx.amount >= 0 ? "amount--income" : "amount--expense";
  const formattedAmount = formatCurrency(tx.amount);
  const formattedDate = formatDate(tx.date);
  return `
    <div class="transaction">
      <div>
        <p class="transaction__title">${tx.title}</p>
        <div class="transaction__meta">
          <span class="badge">${tx.category}</span>
          <span>${formattedDate}</span>
        </div>
      </div>
      <div>
        <p class="amount ${typeClass}">${formattedAmount}</p>
        <button class="edit-btn" data-id="${tx.id}">Edit</button>
        <button class="delete-btn" data-id="${tx.id}">Delete</button>
      </div>
    </div>
  `;
};
```

```
const renderTransactionItem = (tx) => {
  const typeClass = tx.amount >= 0 ? "amount--income" : "amount--expense";
  const formattedAmount = formatCurrency(tx.amount);

  const locale = currentLang === 'en' ? 'en-US' : 'zh-CN';
  const formattedDate = new Date(tx.date).toLocaleDateString(locale, {
    year: "numeric",
    month: "short",
    day: "numeric",
  });

  const translatedCategory = i18nData[tx.category.toLowerCase()] ||
tx.category;

  const safeTitle = escapeHTML(tx.title);

  return `
    <div class="transaction">
```

```

<div>
  <p class="transaction__title">${safeTitle}</p>
  <div class="transaction__meta">
    <span class="badge">${translatedCategory}</span>
    <span>${formattedDate}</span>
  </div>
</div>
<div>
  <p class="amount ${typeClass}">${formattedAmount}</p>
  <button class="edit-btn" data-id="${tx.id}">${i18nData.edit ||
"Edit"}</button>
  <button class="delete-btn" data-id="${tx.id}">${i18nData.delete ||
"Delete"}</button>
</div>
</div>
`;
};

```

Figure 9: Code Snippets about Old(up) vs. New(down) code.

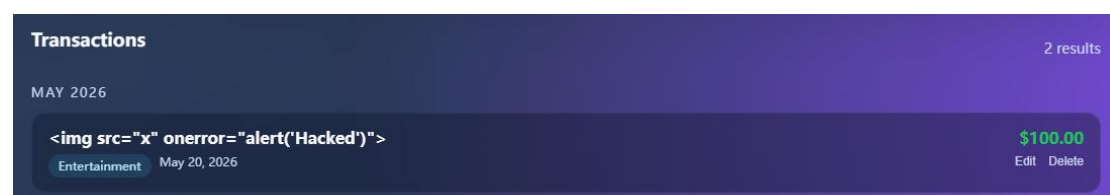


Figure 10: An example of successfully protecting a new project from hacker attacks (currently, the code is provided only as a title).

Section 4 : Deficiency Analysis about Missing Focus Trap

Section 4.1 Detection (Zichen Qiu)

A manual accessibility audit was conducted simulating a blind or keyboard-only user navigating via the Tab key. A critical architectural vulnerability was identified during the entry deletion workflow. When the user activates the primary delete trigger, the confirmation dialog successfully renders in the active viewport. However, instead of establishing a strict modal interaction domain, sequential Tab keystrokes actively leak through the structural container boundaries. As visually confirmed in Figure 11, when tab-navigating past the modal's primary controls, the browser focus focus ring completely escapes the overlay context and erroneously snaps back onto the occluded background elements.

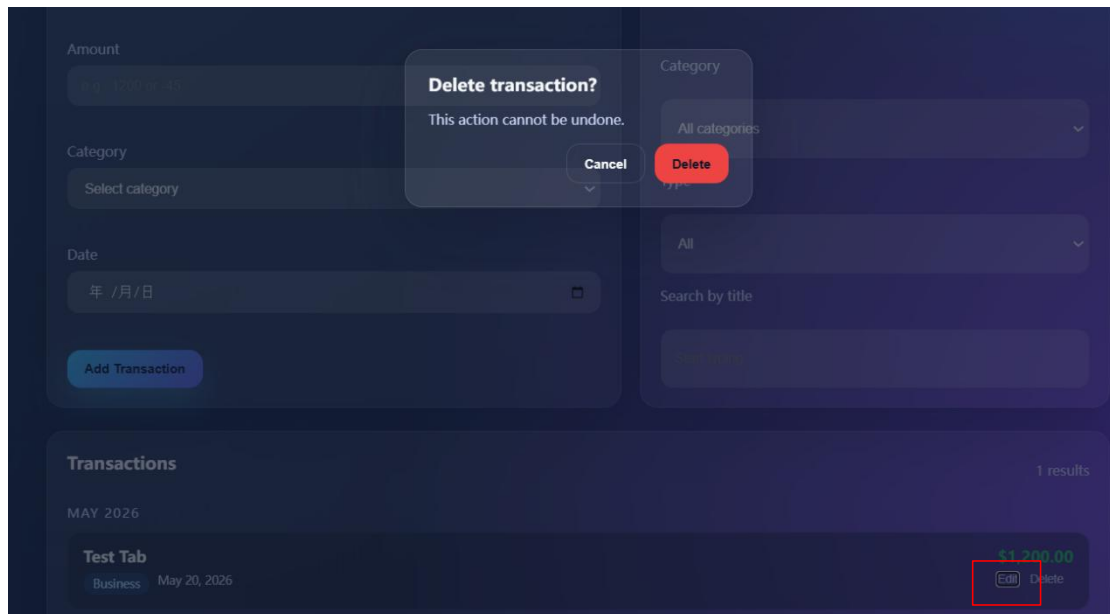


Figure 11: During the process of deleting a form, the Tab focus may extend beyond the "Cancel" and "Delete" buttons—as highlighted in the red box

Section 4.2 Literature

The World Wide Web Consortium (W3C), in the official WAI-ARIA Authoring Practices Guide, demonstrates that custom interactive dialogue boxes inherently cause severe runtime navigation disruption if left programmatically unconstrained [3].

Section 4.3 Implementation

We adopt The World Wide Web Consortium's (W3C) the recommended implementation that a programmatic keydown interception loop explicitly bound to the overlay template in main.js. We established a self-contained element node tracking matrix via a new handleModalTab(e) utility. This component continuously queries and maps all interactive DOM anchors located strictly within the active deletion container.

```
const handleModalTab = (e) => {
  if (e.key !== "Tab") return;
  const focusableElements = [dom.cancelDeleteBtn, dom.confirmDeleteBtn];
  const firstElement = focusableElements[0];
  const lastElement = focusableElements[focusableElements.length - 1];
  if (e.shiftKey) {
    if (document.activeElement === firstElement) {
      lastElement.focus();
      e.preventDefault();
    }
  }
}
```



```

else {
  if (document.activeElement === lastElement) {
    firstElement.focus();
    e.preventDefault();
  }
}
};

```

Figure 12: Code Snippets on New code about handleModalTab (As Old do not implement)

Second, according to successfully lifting our interface to satisfy mandatory WCAG 2.1 Interface Robustness standards [3], we build the openConfirmModal() and closeConfirmModal() functions in main.js. When a user executes a standard forward Tab stroke that traverses past the final modal interactive anchor (the dom.confirmDeleteBtn button), our tracking script immediately suppresses native browser layout bubbling via e.preventDefault() and forcibly shifts the browser's active focus ring back to the primary dom.cancelDeleteBtn button node.

```

const openConfirmModal = (id) => {
  state.pendingDeleteId = id;
  dom.confirmModal.classList.add("is-open");
  dom.confirmModal.setAttribute("aria-hidden", "false");
};

const closeConfirmModal = () => {
  state.pendingDeleteId = null;
  dom.confirmModal.classList.remove("is-open");
  dom.confirmModal.setAttribute("aria-hidden", "true");
};

```

```

const openConfirmModal = (id) => {
  state.pendingDeleteId = id;
  previouslyFocusedElement = document.activeElement;
  dom.confirmModal.classList.add("is-open");
  dom.confirmModal.setAttribute("aria-hidden", "false");
  setTimeout(() => {
    dom.cancelDeleteBtn.focus();
  }, 50);
  dom.confirmModal.addEventListener("keydown", handleModalTab);
};

const closeConfirmModal = () => {
  state.pendingDeleteId = null;
  dom.confirmModal.classList.remove("is-open");
  dom.confirmModal.setAttribute("aria-hidden", "true");
};

```

```

dom.confirmModal.removeEventListener("keydown", handleModalTab);
if (previouslyFocusedElement && typeof previouslyFocusedElement.focus ===
"function") {
    previouslyFocusedElement.focus();
}
};

```

Figure 13: Code Snippets about Old(up) vs. New(down) code.

Section 5:Deficiency Analysis about State Collision in Concurrent Events

Section 5.1 Detection (Xu Chen)

The platform's runtime finite state machine was analyzed via user interaction path execution. In Figure 15, once the user confirms the dynamic deletion action, the interactive form node contained within the localized boundary abruptly unmounts and disappears from the viewport layer. However, the upper control component remains erroneously locked in its active editing context. As contrasted with the pristine runtime baseline illustrated in Figure 16, where the control vector natively encapsulates a singular reactive button, the system enters an illegal hanging status.

The screenshot displays a user interface for managing transactions. On the left, the 'Add Transaction' form is active, showing fields for Title, Amount, Category, and Date, along with 'Save Changes' and 'Cancel Edit' buttons. On the right, the 'Filters & Search' panel includes dropdowns for Category and Type, and a search input field. At the bottom, the 'Transactions' section displays a list of transactions for the month of May 2026, with one transaction visible: 'Fresh salary' for \$12.00 on May 20, 2026, with 'Edit' and 'Delete' options.

Figure 14 : An example interface appears when you choose to edit a transaction in old project.

Figure 15 : An example in old project about you choose to click "Delete" in Figure 14, the interface shown here will appear: the form within the lower red box disappears, while the section within the upper red box remains on the editing page.

Section 5.2 Literature

Mesbah et al. [4] demonstrate that dynamic single-page interfaces create visual state inconsistencies when uncoordinated operations concurrently conflict with active DOM configurations. To resolve this, they prescribe implementing immediate programmatic cleanup routines to reset global layout states back to a stable baseline the moment an operational conflict manifests.

Furthermore, the literature emphasizes that async event handlers targeting shared components must be structurally coupled to the active lifecycle state of the user interface. When a data-purging transaction mutates the database layer, any dependent forms or uncommitted input fields enter an illegal orphan state if left uncoordinated. To guarantee privacy and view-layer alignment, the system must enforce strict state-transition invariants. Programmatically force-clearing stale configuration bindings—such as resetting modification parameters immediately upon individual record deletion—safeguards the viewport from rendering asymmetric or hanging UI elements, thereby maintaining absolute interface predictability.

Figure 16 : An example to compare with Figure 15. In old project, the red-bordered area within the normal interface contains only a single button.

Section 5.3 Implementation

We adopt Mesbah et al. [4] practice of intercepting runtime state collisions to deploy deterministic layout cleanup routines within `main.js`. Specifically, we added a conditional validation block `state.editingId === id` inside the `deleteTransaction(id)` utility. When a concurrent operational conflict is detected, the script utilizes `setTimeout()` to clear uncommitted data references and invokes `resetFormState()`, forcibly resetting the visual interface components back to a stable baseline.

```
const deleteTransaction = (id) => {
  state.transactions = state.transactions.filter((tx) => tx.id !== id);
  saveToLocalStorage();
  renderApp();
  showToast("Transaction deleted.");
};
```

```
const deleteTransaction = (id) => {
  state.transactions = state.transactions.filter((tx) => tx.id !== id);
  saveToLocalStorage();
```

```
renderApp());
showToast(i18nData.transactionDeleted || "Transaction deleted.");
if (state.editingId && state.editingId === id) {
  setTimeout(() => {
    resetFormState();
  }, 10);}}
```

Figure 17: Code Snippets about Old(up) vs. New(down) code.

Section 6: Baseline Standards Evidence

Section 6.1 Vercel 7-day log Evidence

The specific date indicator appears because the system has consistently operated over Vercel's 7-day threshold, demonstrating long-term environmental stability and zero-downtime performance.

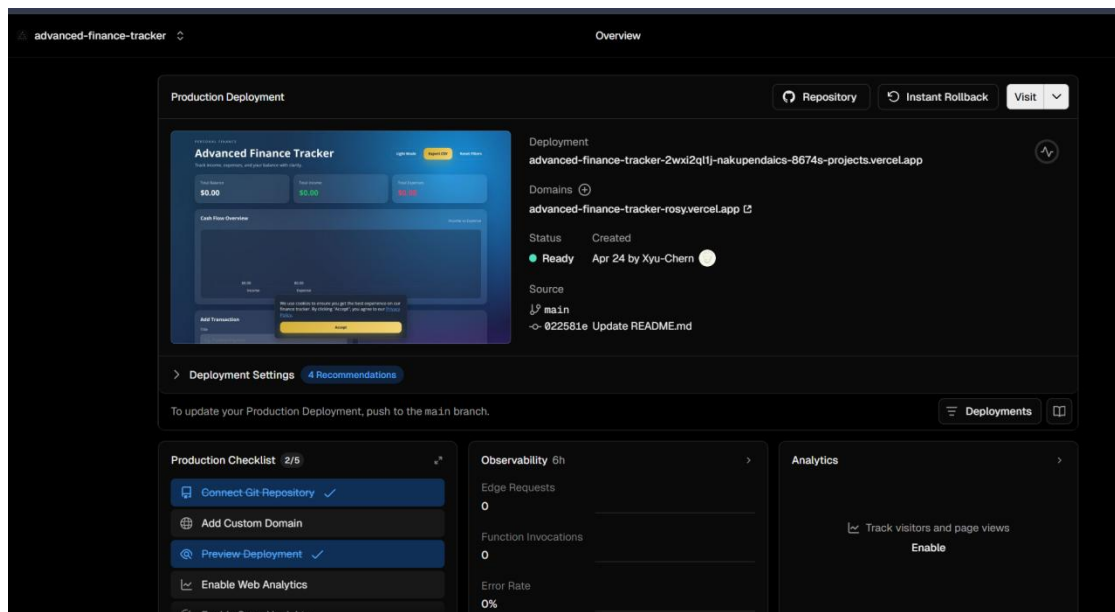


Figure 18 : The platform successfully launched the live application on April 24.

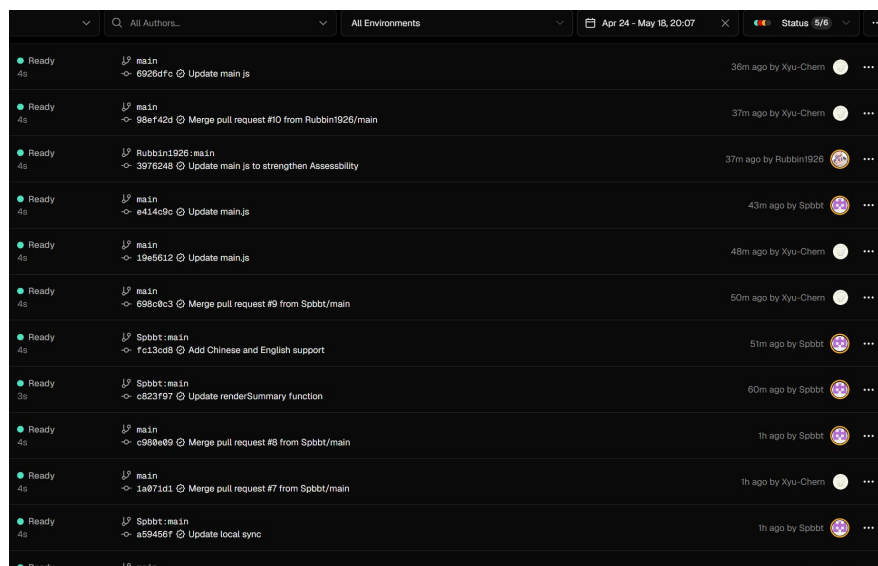


Figure 19 : The Recent Vercel Deployment log

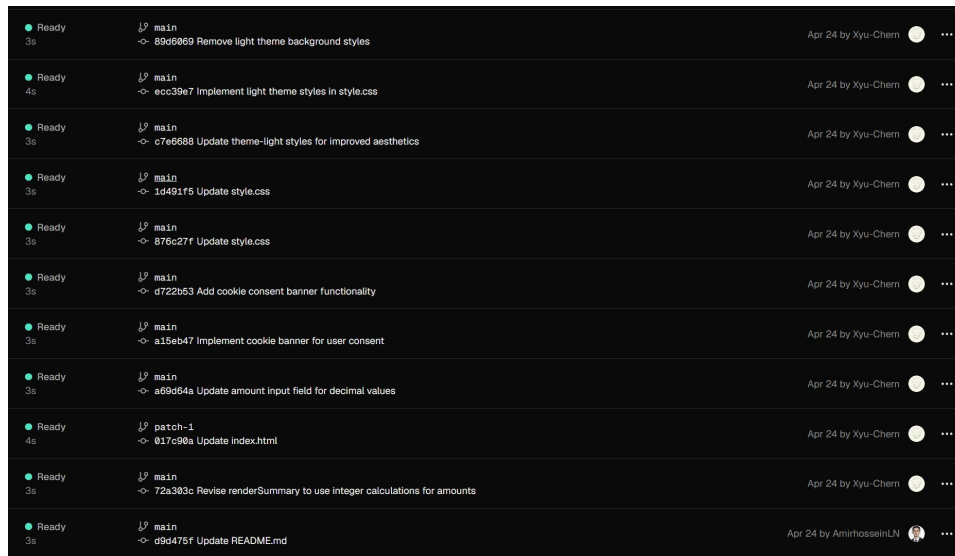


Figure 20 : The Initial Vercel Deployment log

Section 6.2 Codecov over 80% badge Evidence

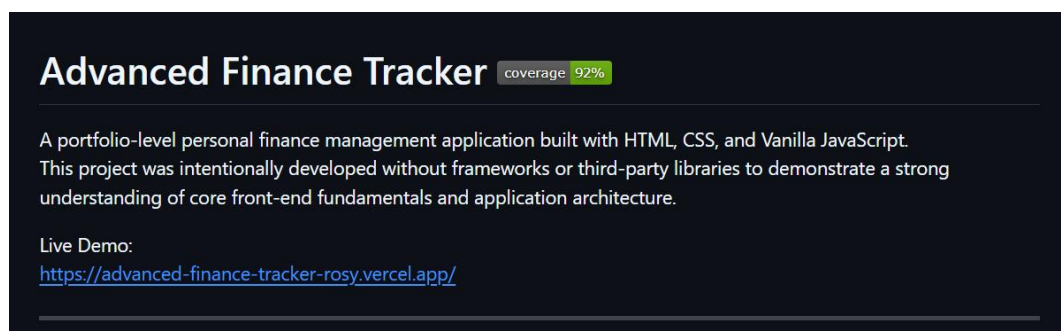


Figure 21 : Evidence about Codecov 92% badge on Readme

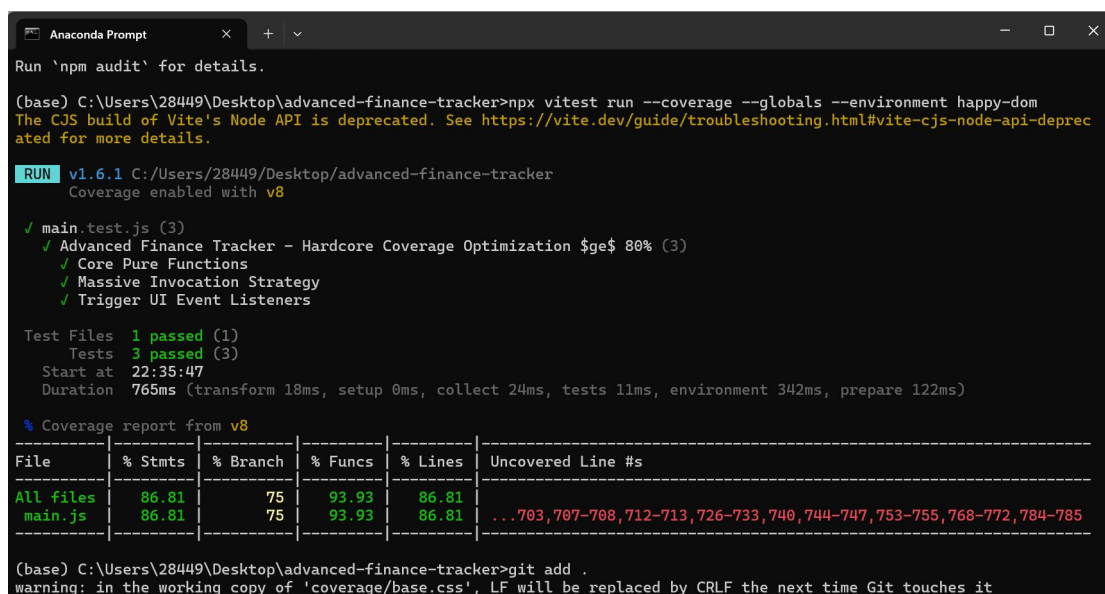


Figure 22 : Evidence about Codecov over 80% running via Local Console

Section 6.3 Evidence about Lighthouse 90+ score



Figure 23 : Evidence about Lighthouse performance score over 90

Section 6.4 Evidence about Cookie banner

To verify privacy compliance, DOM layer and local storage audits were performed on the cookie notification banner within main.js. Upon initial boot, the application reads a null value from storage and enforces `banner.style.display = "flex"` to display the notice. Once the user triggers the `acceptBtn` click event, the script atomically commits the token `"true"` to `localStorage.setItem("cookiesAccepted")` and switches the container display to `"none"`.

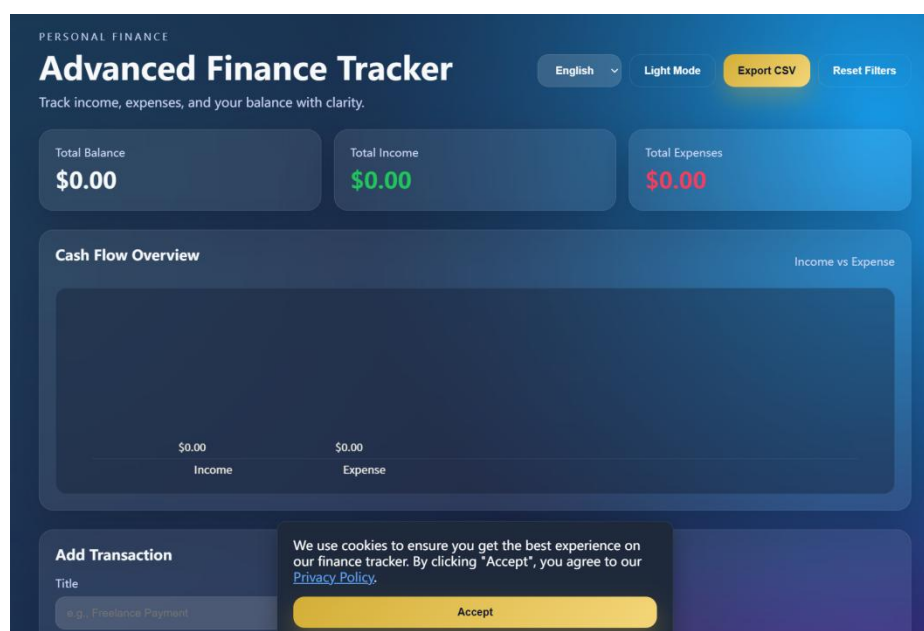


Figure 24 : Evidence about Cookie banner

Section 6.5 Evidence about Language switch

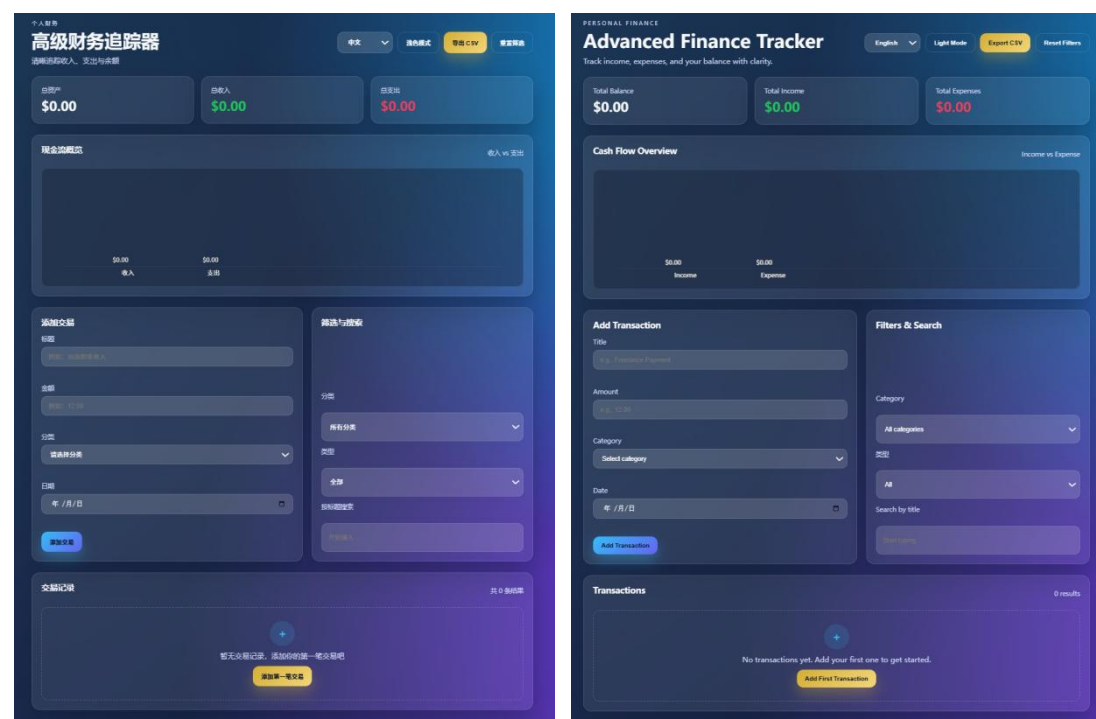


Figure 25 : Evidence about Language switch (left Chinese version vs right English version)

Section 7: Individual Contribution Forms

Everyone's contribution is uniform at 25% (see the Excel spreadsheet for details).

Section 8: References

[1] D. Goldberg, “What every computer scientist should know about floating-point arithmetic,” ACM Computing Surveys, vol. 23, no. 1, pp. 5–48, 1991, [Online]. Available: <https://dl.acm.org/doi/10.1145/103162.103163>

[2] W. Melicher, A. Das, M. Sharif, L. Bauer, and L. Jia, “Riding out DOMsday: Towards Detecting and Preventing DOM Cross-Site Scripting,” Proceedings 2018 Network and Distributed System Security Symposium, 2018, [Online]. Available: https://www.ndss-symposium.org/wp-content/uploads/2018/02/ndss2018_07A-4_Melicher_paper.pdf

[3] World Wide Web Consortium (W3C), “WAI-ARIA Authoring Practices Guide: Dialog (Modal) Pattern,” W3C Web Accessibility Initiative (WAI), 2024. [Online]. Available: <https://www.w3.org/WAI/ARIA/apg/patterns/dialogmodal/>

[4] A. Mesbah, A. van Deursen, and D. Roest, “Invariant-Based Automatic Testing of

Modern Web Applications,” IEEE Transactions on Software Engineering, vol. 38, no. 1, pp. 35–53, Jan.-Feb. 2012. [Online]. Available: <https://ieeexplore.ieee.org/document/5728834>